

Anonymous Types Lambdas

CS-3180

Classes

- Java convention is one class-per file
- + a package system
- Python has similar organization rules
- "Group things logically"



Interfaces Parameters

- OOP is often rigid
- Adding a class between a parent and child
- Painful to refactor
- Interfaces are often a better choice for libraries

```
public static void libraryFunc(Interface a){  
    a.method()  
}
```

Rigidity

- Interfaces are not "concrete"
- At least not in the way classes are
- Contain declarations, but no definitions
- Useful as types, poses some other problems

```
interface Runnable {  
    public void run();  
}
```

Boilerplate

- "Lots of code, little logic"
- Consider the `Thread` class
- Accepts a `Runnable` as a parameter
- `.start()` executes the `run()` in the background

```
1 // standard library definition
2 interface Runnable {
3     public void run();
4 }
5
6 Thread t1 = new Thread(
7     [RUNNABLE INSTANCE]
8 )
9
10 t1.start()
11
```

Interface Instances

- [RUNNABLE INSTANCE]
- Really means:
- A class that implements `Runnable`
- Well lets do that then:
- Have it sleep for `N` seconds
- See the consequence

Interface instances

- Can't alter contract, need N as constructor param

```
1 class Task implements Runnable {
2     private int sleepForSeconds;
3
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
```

Interface instances

- Can't alter contract, need N as constructor param

```
1 class Task implements Runnable {
2     private int sleepForSeconds;
3
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
```


Interface instances

- Can't alter contract, need N as constructor param

```
1 class Task implements Runnable {
2     private int sleepForSeconds;
3
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
}
```

Interface instances

- Can't alter contract, need N as constructor param

```
1 class Task implements Runnable {
2     private int sleepForSeconds;
3
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
}
```

Interface instances

- Can't alter contract, need N as constructor param

```
1 class Task implements Runnable {
2     private int sleepForSeconds;
3
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
17 }
```

Interface instances

- Can't alter contract, need N as constructor param

```
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
17 }
18
19 public class AnonTypes1 {
20     public static void main(String[] args) {
```

Interface instances

- Can't alter contract, need N as constructor param

```
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
17 }
18
19 public class AnonTypes1 {
20     public static void main(String[] args) {
21         Runnable task = new Task(5);
22
23         Thread t = new Thread(task);
24
25         t.start();
26     }
27 }
```

Interface instances

- Can't alter contract, need N as constructor param

```
1 class Task implements Runnable {
2     private int sleepForSeconds;
3
4     public Task(int sleepForSeconds){
5         this.sleepForSeconds = sleepForSeconds;
6     }
7
8     @Override
9     public void run(){
10         try {
11             Thread.sleep(sleepForSeconds * 1000);
12         } catch (InterruptedException e) {
13             System.out.println("Something went wrong");
14         }
15         System.out.println("Done sleeping!");
16     }
}
```

Carpal Tunnel

- My hands already hurt
- There basically 1? 2? lines of logic
- 30 SLOC
- Kind of ridiculous
- There is some better options

Anonymous Interfaces

- Java (+ others) will let you create interface implementation
- Without a separate class
- No class = No name
- It can't easily be re-instantiated
- Let's see that

Anon. Inner Classes

- Use { } after instantiation (of interface, directly)
- Dropping some try/catch boilerplate

```
1 public class AnonTypes2 {
2     public static void main(String[] args) {
3
4         Runnable task = new Runnable() {
5
6             @Override
7             public void run() {
8                 try {
9                     Thread.sleep(5 * 1000);
10                } catch (Exception e) {}
11
12                System.out.println("Done sleeping!");
13            }
14        };
15    }
16 }
```

Anon. Inner Classes

- Use { } after instantiation (of interface, directly)
- Dropping some try/catch boilerplate

```
1 public class Anonymous {
2     public static void main(String[] args) {
3
4         Runnable task = new Runnable() {
5
6             @Override
7             public void run() {
8                 try {
9                     Thread.sleep(5 * 1000);
10                } catch (Exception e) {}
11
12                System.out.println("Done sleeping!");
13            }
14        };
15
16        Thread t = new Thread(task);
```

Anon. Inner Classes

- Use { } after instantiation (of interface, directly)
- Dropping some try/catch boilerplate

```
4         Runnable task = new Runnable() {
5
6             @Override
7             public void run() {
8                 try {
9                     Thread.sleep(5 * 1000);
10                } catch (Exception e) {}
11
12                System.out.println("Done sleeping!");
13            }
14        };
15
16        Thread t = new Thread(task);
17        t.start();
18    }
```

Anon. Inner Classes

- Use { } after instantiation (of interface, directly)
- Dropping some try/catch boilerplate

```
1 public class AnonTypes2 {  
2     public static void main(String[] args) {  
3  
4         Runnable task = new Runnable() {  
5  
6             @Override  
7             public void run() {  
8                 try {  
9                     Thread.sleep(5 * 1000);  
10                } catch (Exception e) {}  
11  
12                System.out.println("Done sleeping!");  
13            }  
14        };  
15    }  
16 }
```

Anon. Inner Classes

- Can implement multiple methods within { }
- Best when only *used* once
- Implementation is all we care about
- Execute implementation later
- Still a lot of boilerplate
- Can we do better?

Lambda Expressions

- (params) -> { implementation }
- Run has no parameters
- As well as no return
- Could call `.run()`, but Java calls that under the hood

```
1 Runnable task = () -> {  
2     try {  
3         Thread.sleep(5 * 1000);  
4     } catch (Exception e){}  
5 };  
6  
7 Thread t = new Thread(task);  
8 t.start();  
9
```

Lambda Expressions

- Parameters do not need a type
- Static Typing -> Java will infer the types
- Only works for single-method interfaces
- Call those "Functional Interfaces"

```
1 interface Drivable {  
2     public void drive(int distance);  
3 }  
4  
5 public class Lambdas {  
6     public static void main(String[] args) {  
7  
8         Drivable d = (distance) -> {  
9             System.out.println("Drives "  
10                 + distance  
11                 + " miles");  
12         };  
13     }
```

Lambda Expressions

- Parameters do not need a type
- Static Typing -> Java will infer the types
- Only works for single-method interfaces
- Call those "Functional Interfaces"

```
1 interface Drivable {
2     public void drive(int distance);
3 }
4
5 public class Lambdas {
6     public static void main(String[] args) {
7
8         Drivable d = (distance) -> {
9             System.out.println("Drives "
10                 + distance
11                 + " miles");
12         };
13 }
```


Lambda Expressions

- Parameters do not need a type
- Static Typing -> Java will infer the types
- Only works for single-method interfaces
- Call those "Functional Interfaces"

```
3 }
4
5 public class Lambdas {
6     public static void main(String[] args) {
7
8         Drivable d = (distance) -> {
9             System.out.println("Drives "
10                 + distance
11                 + " miles");
12         };
13
14         d.drive(5);
15     }
16 }
```

Lambda Expressions

- Parameters do not need a type
- Static Typing -> Java will infer the types
- Only works for single-method interfaces
- Call those "Functional Interfaces"

```
1 interface Drivable {  
2     public void drive(int distance);  
3 }  
4  
5 public class Lambdas {  
6     public static void main(String[] args) {  
7  
8         Drivable d = (distance) -> {  
9             System.out.println("Drives "  
10                 + distance  
11                 + " miles");  
12         };  
13     }
```

Behavior

- We can assign variables to lambdas
- Behavior, as a variable
- Pass these around throughout our program
- Execute on demand
- Abuse of OOP
- Behavior as data is inherently an idea from the Functional Paradigm

Non-First Class Citizens

- We have created an object to sit in place of a function
- Java has to jump through a lot of hoops for this
- There is no type for a Function
- Wrap as an object instead
- In Java, functions are not "First-Class"

Python Function Types

- Python has a type for functions
- `add` is just a variable
- That you **can** call
- You don't strictly have to

```
1 def add(a, b):  
2     return a + b  
3  
4 # add is reference to its implementation  
5 # we dont actually have to call it with (...)  
6 print(add)  
7  
8 print(add(1, 1))  
9
```

First-Class Citizens

- We can pass these function types around
- (Type hinting becomes tricky)
- Work just like data, that we can ()

```
1 def add(a, b):  
2     return a + b  
3  
4 def mult(a, b):  
5     return a * b  
6  
7 def perform_operation(fn, a, b):  
8     return fn(a, b)  
9  
10 result = perform_operation(add, 3, 4)  
11 print(result)  
12  
13 result = perform_operation(mult, 3, 4)  
14 print(result)
```

First-Class Citizens

- We can pass these function types around
- (Type hinting becomes tricky)
- Work just like data, that we can ()

```
1 def add(a, b):  
2     return a + b  
3  
4 def mult(a, b):  
5     return a * b  
6  
7 def perform_operation(fn, a, b):  
8     return fn(a, b)  
9  
10 result = perform_operation(add, 3, 4)  
11 print(result)  
12  
13 result = perform_operation(mult, 3, 4)  
14 print(result)
```

First-Class Citizens

- We can pass these function types around
- (Type hinting becomes tricky)
- Work just like data, that we can ()

```
1 def add(a, b):  
2     return a + b  
3  
4 def mult(a, b):  
5     return a * b  
6  
7 def perform_operation(fn, a, b):  
8     return fn(a, b)  
9  
10 result = perform_operation(add, 3, 4)  
11 print(result)  
12  
13 result = perform_operation(mult, 3, 4)  
14 print(result)
```


First-Class Citizens

- We can pass these function types around
- (Type hinting becomes tricky)
- Work just like data, that we can ()

```
1 def add(a, b):  
2     return a + b  
3  
4 def mult(a, b):  
5     return a * b  
6  
7 def perform_operation(fn, a, b):  
8     return fn(a, b)  
9  
10 result = perform_operation(add, 3, 4)  
11 print(result)  
12  
13 result = perform_operation(mult, 3, 4)  
14 print(result)
```

First-Class Citizens

- We can pass these function types around
- (Type hinting becomes tricky)
- Work just like data, that we can ()

```
1 def add(a, b):  
2     return a + b  
3  
4 def mult(a, b):  
5     return a * b  
6  
7 def perform_operation(fn, a, b):  
8     return fn(a, b)  
9  
10 result = perform_operation(add, 3, 4)  
11 print(result)  
12  
13 result = perform_operation(mult, 3, 4)  
14 print(result)
```

First-Class Citizens

- We can pass these function types around
- (Type hinting becomes tricky)
- Work just like data, that we can ()

```
1 def add(a, b):  
2     return a + b  
3  
4 def mult(a, b):  
5     return a * b  
6  
7 def perform_operation(fn, a, b):  
8     return fn(a, b)  
9  
10 result = perform_operation(add, 3, 4)  
11 print(result)  
12  
13 result = perform_operation(mult, 3, 4)  
14 print(result)
```

First-Class Citizen

- This property of being able to pass functions as arguments:
- Call them First-Class functions
- One of the fundamental ideas of functional programming

Python Lambda Expression

- Lambda Expressions are not unique to OOP hacks
- Universally applicable
- Python: `lambda params: expression`

```
1 def perform_operation(fn, a, b):
2     return fn(a, b)
3
4 result = perform_operation(
5     lambda a, b: a - b,
6     5, 3
7 )
8
9 print(result)
10
```

Python Lambda Expression

- Lambda Expressions are not unique to OOP hacks
- Universally applicable
- Python: `lambda params: expression`

```
1 def perform_operation(fn, a, b):  
2     return fn(a, b)  
3  
4 result = perform_operation(  
5     lambda a, b: a - b,  
6     5, 3  
7 )  
8  
9 print(result)  
10
```

Python Lambda Expression

- Lambda Expressions are not unique to OOP hacks
- Universally applicable
- Python: `lambda params: expression`

```
1 def perform_operation(fn, a, b):  
2     return fn(a, b)  
3  
4 result = perform_operation(  
5     lambda a, b: a - b,  
6     5, 3  
7 )  
8  
9 print(result)  
10
```

Python Lambda Expression

- Lambda Expressions are not unique to OOP hacks
- Universally applicable
- Python: `lambda params: expression`

```
1 def perform_operation(fn, a, b):
2     return fn(a, b)
3
4 result = perform_operation(
5     lambda a, b: a - b,
6     5, 3
7 )
8
9 print(result)
10
```


Python Lambda Expression

- Lambda Expressions are not unique to OOP hacks
- Universally applicable
- Python: `lambda params: expression`

```
1 def perform_operation(fn, a, b):
2     return fn(a, b)
3
4 result = perform_operation(
5     lambda a, b: a - b,
6     5, 3
7 )
8
9 print(result)
10
```

Summary

- Anonymous Types are commonly used to store behavior
- Different languages need to jump through different hoops to do this
- Functions as types = First Class Functions
- Not all languages support this
- Lambda Expression: Quick, anonymous functions

Questions?